

Logic

1 PROPOSITIONS & CONNECTIVES

proposition - a statement that is either true or false ("the list is empty", "x > 5" for a known x); not questions, commands, opinions. In code, any boolean expression **the connectives** - and (both), or (at least one, inclusive), not (flip), implies, iff (both the same). JS: &&, ||, !; iff for booleans is a === b, xor (exactly one) is a !== b

implies (A -> B) - false in exactly one case: A true and B false. A promise about when A holds; A false = promise never tested = true. Not causation

implication as code - A -> B rewrites to !A || B. "every admin must be verified" is !isAdmin || isVerified; the enforcing guard is the rule's negation: if (isAdmin && !isVerified) throw

truth table - one row per input combination, 2^n rows for n booleans. Two conditions are interchangeable exactly when their tables match on every row - the refactoring safety net

De Morgan's laws - not(A and B) = (not A) or (not B); not(A or B) = (not A) and (not B). Negating !(isValid && isReady) is !isValid || !isReady, not !isValid && !isReady

if-then's three relatives - "if A then B" (A->B); converse B->A (NOT equivalent), inverse notA->notB (NOT equivalent), contrapositive notB->notA (ALWAYS equivalent). Only the contrapositive matches the original's truth

vacuous truth - "if A then B" is true whenever A is false, regardless of B. [].every(x=>x>1000)===true; "all users in this empty segment are premium" is technically true, practically meaningless

necessary vs sufficient - necessary: required but not enough alone (a ticket to board a flight). Sufficient: enough alone but not the only way (born in the US, for citizenship). In arrows: A sufficient for B is A->B; A necessary for B is B->A

2 QUANTIFIERS & SETS

predicate - a proposition with holes: isEven(x) is not true or false until x is filled in. In code, any function returning a boolean **all (forall)** - every x satisfies P; arr.every(p). True on an empty set. One counterexample kills it; no number of confirming examples proves it for an infinite set

some (exists) - at least one x satisfies P; arr.some(p). False on an empty set. One example proves it

negating quantifiers - "all X are Y" negates to "some X are not Y" (not "no X are Y"). "some X are Y" negates to "no X is Y". "all requests succeeded" negates to "at least one failed". In code: !arr.every(p) = arr.some(x => !p(x))

set - unordered, no duplicates, membership is the only question. Subset: every element of A is also in B: [...a].every(x => b.has(x))

set operations - union (in either), intersection (in both), difference A minus B (in A, not in B). Set-builder "all x in S where P(x)" is arr.filter(p). "users with access they should not have" is actual minus allowed

3 SIMPLIFYING CONDITIONALS

refactor against the truth table - list the 2^n rows and outcomes before rewriting; the rewrite must match on all rows. Catches !(a && b) "simplified" to !a && !b (differs when exactly one is true)

factoring out - (A && B) || (A && C) = A && (B || C), pull the shared term out. Most real cleanups of long conditions are this law applied once or twice

merging branches - two branches with the same body merge on OR: if (a) return f(); if (b) return f(); is if (a || b) return f()

absorption and constants - A || (A && B) = A; A && true = A; A || true = true; A && false = false. Pays off after inlining a feature flag: mechanically deletes the dead branches **exactly one of two** - (A || B) && !(A && B), for booleans just A !== B. Requirements "either A or B" usually means exactly-one; code || means at-least-one - the mismatch is a requirements bug

4 PROPERTY-BASED TESTING

example vs property test - example pins one input to one output (sort([3,1,2]) is [1,2,3]); a property states a rule for all inputs ("output is ordered") and fast-check fires hundreds of random inputs at it

the four property sources - (1) output invariants: ordered, same length, same elements; (2) round trip: decode(encode(x)) = x; (3) oracle: match a slow obviously-right version; (4) relations between calls: total(cartWithItem) = total(cart) + price

shrinking - a failing random input is automatically reduced to the smallest input that still fails, plus a replay seed - the report is [0, -1], not a 40-element array

a property test is only as good as its properties - a "sort" returning the input untouched passes "same length, same elements"; only "output is ordered" kills it. Choosing properties is specification; weak properties fail silently

5 CONTRACTS & API COMPATIBILITY

precondition / postcondition - pre: what must be true at the call (caller's job); post: what is guaranteed at return (function's job). withdraw(amount): pre amount > 0 && amount <= balance; post newBalance = oldBalance - amount

contracts as asserts - preconditions asserted at the top, postconditions before return. The payoff is blame: failed pre = caller's bug, failed post = the function's own

the substitution rule - a replacement (subclass, v2, mock) may accept more (weaker pre) and promise more (stronger post), never the reverse. Liskov substitution said in contract language

backwards-compatible API evolution - v2 must accept every call v1 accepted (optional field made required = breaking) and keep every promise v1 made (removed response field = breaking). Compatible = pre only weakens, post only strengthens

6 PROVING CODE CORRECT

Hoare triple - {P} code {Q}; if P holds before, Q holds after. A contract written for proving; tools like Dafny prove it for all inputs instead of asserting per run

loop invariant - true before the loop, preserved by every pass; at exit, invariant + exit condition give the result. Summing: sum equals total of arr[o..i-1] at the top of each pass; at exit i = arr.length

7 LOGIC IN DATABASES

a query is a predicate - a table is a set of facts, WHERE P returns exactly the rows where P is true, a join is an AND across two tables

NULL is unknown (three-valued logic) - any comparison with NULL is unknown, even NULL = NULL. WHERE keeps only true (drops false AND unknown); NOT unknown is unknown. x NOT IN (1, NULL) returns nothing, ever. Fixes: IS NULL, IS NOT NULL, COALESCE

constraints are invariants - UNIQUE, NOT NULL, FOREIGN KEY, CHECK are claims the database re-verifies on every write and refuses to let become false; safer there than in application code

8 DECISION TABLES

decision table - one column per condition, one row per combination, one outcome per row; n booleans = exactly 2^n rows

gaps and contradictions - a combination with no row is a requirements gap (never decided); two rows with same inputs and different outcomes is a contradiction. Enumerate, then ask the product owner one blank cell at a time

don't-care values and the test plan - when an outcome ignores a condition, mark it - and merge rows. The finished table is a test plan: one test per row is complete coverage of the condition space

9 SOLVERS

the declarative flip - imperative code says how to build the answer; a solver takes what a valid answer looks like (variables, allowed values, constraints) and searches. Use when rules are easy to state and the procedure is hard

constraint solving (the suitcase) - items with weight and value, capacity 10kg: one yes/no variable per item, weights sum <= 10, maximize value sum. Tools: MiniZinc, OR-Tools. Same shape: scheduling, seat assignment

SAT and SMT - SAT: find a true/false assignment satisfying a boolean formula. SMT = SAT + theories (integers, arrays) so x + y < 10 is stated directly; Z3 is the standard. Runs today in package version resolution

10 LOGIC PROGRAMMING

Prolog - facts (parent(tom, bob).) and rules (grandparent(X, Z) :- parent(X, Y), parent(Y, Z). where :- reads "if"); a query searches for every derivable answer, and the same rule runs in any direction

Datalog - Prolog restricted so every query terminates; the query language inside static analyzers (CodeQL) and Datomic. Practical read: recursive queries over facts - "everything reachable from X" in two lines